

Resenha Crítica: Verificação e Validação de Software

Aplicação prática ao cálculo de Fibonacci de alta precisão e análise de literatura acadêmica

Autores: Juliano Machado e Renan Bertoletti | **Professor:** Prof. Dr. Marco Aurélio Souza Mangan

Data: Semestre 2026/I | **SUT:** The Millionth Fibonacci Kata (Codewars, 3 kyu)

1. Introdução e Caracterização do Objeto de Estudo

A garantia de qualidade é parte essencial da Engenharia de Software: o objetivo é ter certeza de que o sistema funciona corretamente em diferentes situações de uso. Este trabalho aplica técnicas de Verificação e Validação (V&V) a um problema que exige muito do computador: o desafio *The Millionth Fibonacci Kata*, da plataforma Codewars (nível 3 kyu).

O Sistema Sob Teste (SUT) deve calcular o valor exato de $\text{fib}(n)$, da sequência de Fibonacci, para n inteiro entre -2.000.000 e 2.000.000. Há dois desafios: lidar com números de centenas de milhares de dígitos e calcular corretamente as posições negativas.

1.1 Linguagem, plataforma e estrutura do SUT

Usamos Java (JDK 17+), JUnit 5 e Apache Maven. Em Java, os tipos `int` e `long` estouram o limite a partir de $n = 93$ ($\text{fib}(92)$ ainda cabe em um `long`). Por isso é obrigatório usar `java.math.BigInteger`, que não perde precisão. O método recursivo simples é muito lento (tempo exponencial) e até o método iterativo (n passos) fica pesado com milhões de iterações. A solução usa o algoritmo *Fast Doubling*, que calcula $\text{fib}(n)$ com cerca de $\log|n|$ passos, e responde rápido mesmo para $n = 2.000.000$.

1.2 Regras de negócio: índices negativos

Partindo de $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ e invertendo a conta, chega-se a $\text{fib}(-n) = (-1)^{n+1} \cdot \text{fib}(n)$. Na prática: para índices negativos ímpares (-1, -3, -5) o resultado é igual ao positivo; para índices negativos pares (-2, -4, -6) o valor é o mesmo, mas com sinal trocado. Acertar essa regra é um requisito funcional crítico para os testes.

2. Referencial Teórico: Síntese Crítica dos Artigos

Foram estudados dois artigos de conferências importantes da área de teste automatizado (ISSTA e ICSE).

2.1 Artigo 1: Echidna — testes automáticos para contratos inteligentes (ISSTA 2020)

Grieco et al. (2020) apresentam o Echidna, ferramenta de código aberto que faz *fuzzing* (testes com entradas geradas automaticamente) em contratos inteligentes da blockchain Ethereum. Antes de testar, ele usa o analisador Slither para extrair informações do contrato (como nomes de funções e constantes) e depois gera sequências de transações para tentar violar asserções e propriedades escritas pelo usuário. Também estima o gasto máximo de "gás" e, quando acha uma falha, reduz a sequência de chamadas até a menor que ainda reproduz o erro (*shrinking*).

Pontos fortes: é fácil de instalar e usar (não precisa montar uma blockchain local), dá respostas rápidas e já foi usado em auditorias de segurança reais. **Limitações:** só funciona com contratos Ethereum (Solidity/Vyper) e só encontra problemas nas propriedades que alguém escreveu. **Uso no nosso trabalho:** como o SUT é um programa Java comum, não é possível usar o Echidna como base; ele entrou na resenha apenas para comparar abordagens de teste automático.

2.2 Artigo 2: RLCheck — entradas válidas e diversas com aprendizado por reforço (ICSE 2020)

Reddy et al. (2020) partem de um problema comum no Teste Baseado em Propriedades (PBT): muitos testes só fazem sentido para entradas *válidas* (regra do tipo $P(x) \Rightarrow Q(x)$), mas um gerador aleatório comum produz poucas entradas válidas e, entre elas, quase todas parecidas. Os autores batizam isso de *diversifying guidance problem* (problema de guiar o gerador para produzir muitas entradas válidas e diferentes entre si). Para resolvê-lo, o RLCheck usa aprendizado por reforço simples (controle de Monte Carlo, com uma tabela de valores): o gerador ganha mais pontos por entradas válidas e inéditas e perde pontos por entradas inválidas, aprendendo aos poucos quais escolhas levam a boas entradas. Tudo isso em modo caixa-preta, sem instrumentar o código testado.

Pontos fortes: nos mesmos programas usados pelo Zest (Ant, Maven, Closure Compiler e Rhino), em 5 minutos, gerou de 1,4x a 40x mais entradas válidas e diversas que o QuickCheck e o Zest; a ausência de instrumentação deixa os testes mais rápidos. Os autores também reencontraram alguns bugs já conhecidos (em Ant, Closure e Rhino), mas afirmam que não fizeram um estudo extenso de bugs e que o resultado não é conclusivo; no Rhino, o QuickCheck simples achou mais bugs. **Limitações:** a tabela de aprendizado não escala para geradores muito grandes, então foi preciso limitar as opções; tem dificuldade em achar a primeira entrada válida quando as regras são muito rígidas (caso do Ant); a cobertura de código estaciona cedo, pois o método tende a repetir o que já funcionou; e os parâmetros de recompensa foram ajustados à mão.

Tabela 1: Comparação entre os artigos estudados

Dimensão	Echidna (Grieco et al., 2020)	RLCheck (Reddy et al., 2020)
Domínio	Contratos inteligentes Ethereum (Solidity/Vyper)	Programas Java e testes baseados em propriedades (via JQF)
Abordagem	Fuzzing de transações, com apoio de análise estática (Slither)	Aprendizado por reforço (Monte Carlo) guiando o gerador
Modo	Usa informação de cobertura durante a execução	Caixa-preta (sem instrumentação)
Oráculo (como decide se passou)	Propriedades do usuário, asserções e limites de gás	Propriedade $P(x) \Rightarrow Q(x)$ do teste
Métrica de sucesso	Violações encontradas e rapidez	Quantidade e diversidade de entradas válidas
Principal limitação	Restrito à plataforma Ethereum	Tabela de aprendizado cresce com o gerador

3. Aplicação: Técnicas de V&V no SUT de Fibonacci

O que vimos nos artigos e nas aulas do Prof. Marco Aurélio Mangan virou uma suíte de testes com cinco técnicas:

- **a) Classes de equivalência:** o domínio de n foi dividido em grupos: zero; positivos pequenos (1 a 92); positivos gigantes (93 a 2.000.000); negativos ímpares; negativos pares; e valores inválidos (nulos ou fora dos limites).
- **b) Análise de valor limite (BVA):** testes nas fronteiras, onde os erros costumam aparecer: ao redor de $n = 0, 1, -1, 92, 93$ e do limite de 2.000.000.
- **c) Design por Contrato (DbC):** a pré-condição exige n não nulo e dentro do domínio válido; a pós-condição garante retorno não nulo, em `BigInteger`, com o resultado exato; o invariante garante que o estado da classe não muda.
- **d) Teste baseado em propriedades (PBT):** com o `jqwik`, verificamos regras que valem para qualquer n sorteado, como a simetria $\text{fib}(-n) = (-1)^{n+1} \cdot \text{fib}(n)$ a soma $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ e a coprimaridade de termos vizinhos ($\text{mdc de fib}(n) \text{ e fib}(n+1) \text{ é } 1$), sem precisar de tabelas fixas de valores.
- **e) Teste estrutural e cobertura:** o `JaCoCo` mede se os testes passam por todos os caminhos do código (decisão de sinal, caso zero e recursão do `Fast Doubling`).

Ligação com os artigos: o RLCheck trata de geradores com regras de validade rígidas, algo que quase não existe aqui, já que as entradas do Fibonacci são apenas números inteiros em um intervalo. O que aproveitamos dele foi a ideia central do PBT (testar propriedades $P(x) \Rightarrow Q(x)$ com entradas geradas automaticamente), que no nosso caso é feita pelo `jqwik`. O Echidna não foi usado, pelos motivos da seção 2.1.

4. Discussão

Resultados. Foram implementados 42 testes e todos passaram (0 falhas, 0 erros): 15 de classes de equivalência, 8 de valor limite, 16 de Design por Contrato e 3 propriedades do `jqwik`, cada uma com cerca de 1.000 execuções aleatórias. A cobertura medida pelo `JaCoCo` foi de 97% das instruções e 100% dos ramos; a única instrução não coberta é o construtor padrão implícito da classe, que só tem métodos estáticos e nunca é instanciada. As técnicas se complementam: classes de equivalência e valor limite cobrem casos escolhidos à mão; o PBT cobre muitos valores que não pensaríamos em escrever; e a cobertura mostra se algum trecho ficou sem teste.

Dificuldades. Os testes de propriedade com `BigInteger` ficaram lentos para valores altos, então foi preciso limitar o intervalo sorteado pelo `jqwik` (por exemplo, até 5.000 em vez de 2.000.000) para manter o tempo de execução viável. Outra dificuldade foi definir o resultado esperado para entradas grandes, como `fib(1.000)`: foi necessária uma fonte de referência externa confiável para servir de oráculo.

Ameaças à validade. (1) O kata é simples, com uma única função e entradas numéricas; por isso os resultados podem não valer para sistemas maiores, nem mostram o benefício de técnicas como o RLCheck. (2) Cobertura de 100% dos ramos mostra que o código foi executado, mas não garante que ele esteja certo. (3) Os valores esperados para entradas grandes vêm de uma fonte externa; se ela estivesse errada, os testes aceitariam um resultado errado. (4) O PBT sorteia valores só até 5.000, então não exercita os números gigantes; esses ficam por conta dos testes escolhidos à mão, e mesmo o sorteio pode não pegar justamente o caso problemático. (5) Não executamos Echidna nem RLCheck: a comparação com eles é conceitual, não experimental.

5. Conclusão e Lições Aprendidas

O trabalho ligou os conceitos de V&V com a prática de testar um software exigente. A literatura mostra que a geração automática de testes, seja por fuzzing ou por aprendizado por reforço, está avançando, mas essas ferramentas só rendem bem quando alguém define contratos claros, bons oráculos de teste e uma boa divisão do domínio de entradas. Também aprendemos que cada ferramenta tem seu campo de uso: o Echidna não serve para um programa Java comum, e o RLCheck ajuda mais quando as entradas têm regras de validade difíceis de satisfazer.

6. Referências Bibliográficas (ABNT)

CODEWARS. **The Millionth Fibonacci Kata**. Desafio de programação de nível 3 kyu. Disponível na plataforma Codewars, 2026.

GRIECO, Gustavo *et al.* Echidna: effective, usable, and fast fuzzing for smart contracts. In: ACM SIGSOFT INTERNATIONAL SYMPOSIUM ON SOFTWARE TESTING AND ANALYSIS (ISSTA), 29., 2020. **Proceedings** [...]. [S. l.]: ACM, 2020. p. 557–560. DOI 10.1145/3395363.3404366.

JUNIT TEAM. **JUnit 5 User Guide**. Disponível em: <<https://junit.org/junit5/docs/current/user-guide/>>. Acesso em: 6 out. 2026.

MANGAN, Marco Aurélio Souza. **Diretrizes e materiais de aula da disciplina de Verificação e Validação de Software**. Porto Alegre: PUCRS / Escola Politécnica, 2026.

REDDY, Sameer *et al.* Quickly generating diverse valid test inputs with reinforcement learning. In: INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING (ICSE), 42., 2020. **Proceedings** [...]. [S. l.]: ACM, 2020. p. 1410–1421. DOI 10.1145/3377811.3380399.